

Kịch bản quay video demo — CanvasApp (Nhóm 14 / NT106.Q23.ANTT)

Mục tiêu video: Trình diễn đầy đủ 11 mục rubric (10đ) + 10đ creative/UI trong một mạch quay liên lạc.
Thời lượng đề xuất: 18–22 phút (có thể cắt còn 15p khi edit). **Định dạng:** 1080p 30fps, screen-record + voice-over (mic rời nếu có). **Phong cách trình bày:** vừa thao tác vừa thuyết minh ngắn gọn; mỗi thao tác phải “show + tell” — vừa làm vừa giải thích **đang chứng minh mục rubric nào**.

0. Chuẩn bị trước khi bấm Record

0.1. Phần mềm cần mở sẵn ở Desktop (sắp xếp theo workspace)

Phần mềm	Vị trí trên màn hình	Mục đích
OBS Studio / ShareX	Tray icon	Quay màn hình + voice
Visual Studio 2022	Workspace 1	Show code evidence
XAMPP Control Panel	Tray icon	Start MySQL
Trình duyệt Chrome → http://localhost/phpmyadmin	Workspace 1	Show DB
PowerShell (mở 2 cửa sổ)	Workspace 2	Chạy CLI args, netstat , ipconfig
File Explorer mở sẵn bin\Debug\ của 5 project	Workspace 2	Để khởi .exe tay
Wireshark	Workspace 3	Bắt loopback (chuẩn bị filter sẵn)
Notepad / VSCode	Workspace 3	Mở sẵn file appsettings.json để show config

0.2. Reset trạng thái dữ liệu

- Mở phpMyAdmin → DB [canvasapp](#) → **Import** file [Database/reset.sql](#) (xóa hết user/room cũ).
- Re-import [Database/schema.sql](#) rồi [Database/seed_data.sql](#) để có sẵn 3 user demo ([demo1](#) / [demo2](#) / [demo3](#) — password [123456](#)) + 1–2 phòng mẫu.
- Kiểm tra:** vào bảng [users](#) → đúng 3 row; [rooms](#) → 1–2 row.

0.3. Đóng mọi instance cũ trước khi quay

```
Get-Process CanvasApp.* -ErrorAction SilentlyContinue | Stop-Process -Force
```

0.4. Build solution 1 lần cuối

- Visual Studio → [Ctrl](#) + [Shift](#) + [B](#) → **Build succeeded. 0 Error(s).**

- Confirm có .exe ở:
 - CanvasApp.AuthServer\bin\Debug\CanvasApp.AuthServer.exe
 - CanvasApp.Server\bin\Debug\CanvasApp.Server.exe
 - CanvasApp.LoadBalancer\bin\Debug\CanvasApp.LoadBalancer.exe
 - CanvasApp.Client\bin\Debug\CanvasApp.Client.exe

0.5. Cấu hình OBS Scene

Scene	Bố cục
Scene A — Intro	Slide title + logo nhóm
Scene B — Code view	Full screen VS2022
Scene C — Server pool	4 console (Auth ×2, Canvas ×2) + LB ở giữa, layout 2×3
Scene D — Client demo	3 Client side-by-side, chiếm 100% màn hình
Scene E — DB / Wireshark / netstat	Browser + terminal split

0.6. Cấu hình Wireshark trước

- Mở **Adapter for loopback traffic capture**.
- Display Filter mặc định:

```
tcp.port in {9000 9001 9002 9003 9011 9102 9103}
```

- Tô màu: `tcp contains "DRAW_"` → đỏ; `tcp contains "AUTH_"` → xanh; `tcp contains "CHAT_"` → vàng.

0.7. Script narrate sẵn các từ chốt

- “**Đây là bằng chứng cho mục...**” (lặp lại mỗi lần đổi rubric item)
- “**Quan sát console...**”, “**Quan sát DB...**”, “**Quan sát Wireshark...**”
- “**Code evidence ở file X, dòng Y**” (mở VS, Ctrl+G nhảy dòng)

1. Mở đầu video — 0:00–1:00 (Scene A → B)

Tiêu đề & nhóm thực hiện

CanvasApp — Real-time Collaborative Whiteboard over TCP NT106.Q23.ANTT — Lập trình mạng
căn bản Nhóm 14 — 24520453 / 24521187 / 24521501

Mục lục video

Hiện thị: Slide “Nội dung sẽ trình diễn”

- A. Khởi động hệ thống – 5 server + 3 client
- B. Đăng ký / Đăng nhập / OTP / Quên mật khẩu (C.5)
- C. Vẽ realtime với 11 công cụ (C.1, C.2, C.6, Creative)
- D. Database + restart persistence (C.3)
- E. Multi-thread + Multi-server + LB (C.4, C.7, C.11)
- F. Cryptography (BCrypt + HMAC + AES + Wireshark) (C.8)
- G. Demo LAN/VPN (C.9, C.10)
- H. Wrap-up & checklist

2. Khởi động hệ thống

Cảnh 2.1. Khởi MySQL + show DB

Thao tác:

1. Mở **XAMPP** → click **Start** MySQL (port 3306).
2. Mở phpMyAdmin trong tab Chrome → click DB **canvasapp** → bấm vào bảng **users** → show 3 row seed.
3. Click **rooms** → show 1 room mẫu.

Lời thoại:

“Database MySQL đang chạy ở port 3306. Em đã reset và seed sẵn 3 user demo. Quan sát cột **password_hash** đã được băm Bcrypt cost 12 — đây là bằng chứng đầu cho mục Cryptography.”

Bằng chứng cần show rõ trên video:

- Cột **password_hash** dạng **\$2b\$12\$...** (60 ký tự).
- Tổng 7 bảng: **users**, **rooms**, **room_members**, **draw_actions**, **canvas_snapshots**, **chat_messages**, **email_otp_codes**.

Cảnh 2.2. F5 trong Visual Studio (~40s)

Thao tác:

1. Quay sang VS2022, đảm bảo **Configure Startup Projects** đã set 3 project Start (**AuthServer**, **Server**, **LoadBalancer**) theo đúng thứ tự.
2. Bấm **F5**.
3. Quay 3 console xuất hiện theo thứ tự, zoom vào banner.

Banner kỳ vọng (quay rõ từng cái):

```
AUTH SERVER listening on :9001
```

```
CANVAS SERVER listening on :9002
```

```
LOAD BALANCER listening on :9000
```

```
Auth pool: 1 backend(s)
```

```
Canvas pool: 1 backend(s)
```

Lời thoại:

“F5 trong Visual Studio mở đồng thời 3 server. LoadBalancer khởi sau cùng để 2 backend đã listening — quan sát banner cho thấy LB đã đăng ký 1 Auth + 1 Canvas backend.”

Cảnh 2.3. Khởi 2 instance phụ qua CLI args (~30s)

Thao tác:

1. Mở PowerShell ở `CanvasApp.AuthServer\bin\Debug\` → gõ:

```
.\CanvasApp.AuthServer.exe 9011
```

2. Mở PowerShell thứ 2 ở `CanvasApp.Server\bin\Debug\` → gõ:

```
.\CanvasApp.Server.exe 9003
```

3. Quay sang console LoadBalancer → đợi $\leq 5s$ → quay log:

```
[HEALTH] Auth 127.0.0.1:9011 -> UP (recovered)
[HEALTH] Canvas 127.0.0.1:9003 -> UP (recovered)
```

4. Đợi 15s → quay tiếp `[POOL]` snapshot:

```
[POOL]
Auth 127.0.0.1:9001 [UP] conns=0/100 rooms=0 fails=0
Auth 127.0.0.1:9011 [UP] conns=0/100 rooms=0 fails=0
Canvas 127.0.0.1:9002 [UP] conns=0/100 rooms=0 fails=0
Canvas 127.0.0.1:9003 [UP] conns=0/100 rooms=0 fails=0
```

Lời thoại:

“Port server được đọc từ tham số dòng lệnh `args[0]`. Em chạy thêm 2 instance: Auth thứ hai port 9011, Canvas thứ hai port 9003. Sau 5 giây — đúng 1 chu kỳ health check — LB phát hiện 2 backend mới và mark **UP (recovered)**. Đây là bằng chứng đầu tiên cho mục **Multi-Server** và **Load Balancing**.”

Cảnh 2.4. netstat confirm 7 port LISTENING (~20s)**Thao tác:**

1. PowerShell → gõ:

```
netstat -ano | Select-String "LISTENING" | Select-String  
"9000|9001|9002|9003|9011|9102|9103"
```

2. Quay rõ kết quả (zoom in nếu cần) — phải có **7 dòng**.

Bằng chứng:

- 9000 (LB), 9001 (Auth1), 9002 (Canvas1), 9003 (Canvas2), 9011 (Auth2), 9102 (peer-Canvas1), 9103 (peer-Canvas2).

Lời thoại:

“Đây là 7 port đang lắng nghe ở mức OS — 1 LB, 2 Auth, 2 Canvas client-port, và 2 peer-port cho peer mesh giữa các Canvas Server. Tổng cộng 5 process server đang chạy đồng thời.”

3. Đăng ký + OTP + Quên mật khẩu (C.5 — 0.5đ) — 2:30–4:30 (Scene D)**Cảnh 3.1. Code evidence (~20s)**

Thao tác: Trong VS2022, mở nhanh 3 tab (Ctrl+T tìm file), quay lướt từng tab khoảng 5s:

File	Highlight
CanvasApp.AuthServer/Services/OtpService.cs	Hash BCrypt cost 10, TTL 5min, rate-limit method
CanvasApp.AuthServer/UserStore.cs	HashPassword(pwd, 12) + IssueToken HMAC-SHA256
CanvasApp.Client/Forms/RegisterForm.cs	Gửi AUTH_SEND_OTP rồi AUTH_REGISTER kèm OTP

Lời thoại:

“Server side dùng `BCrypt.Net.BCrypt.HashPassword` cost 12 cho mật khẩu, cost 10 cho mã OTP. OTP có TTL 5 phút, single-use và rate-limit 1 lần/phút, 5 lần/giờ cho mỗi email.”

Cảnh 3.2. Đăng ký user mới với OTP (~60s)

Thao tác:

1. Khởi Client #1: chuột phải project **CanvasApp.Client** → Debug → **Start New Instance**.
2. Trên **LoginForm** → click tab **Đăng ký**.
3. Nhập:
 - Username: **videodemo**
 - Email: **<email_thật_của_em>@gm.uit.edu.vn**
 - Password: **123456**
4. Click **Gửi mã OTP** → quay phản hồi UI.
5. Quay sang console **AuthServer 9001** → quay log:

```
[OTP] Sent 6-digit code to <email>, expires in 300s
```

6. (Nếu dev mode in OTP plaintext ra console — nói luôn): zoom vào số OTP đang in.
7. Mở email thật trong tab khác (hoặc dùng OTP console) → copy số OTP.
8. Quay sang Client → nhập OTP → submit → quay UI báo **Đăng ký thành công**.
9. Quay sang phpMyAdmin → refresh bảng **users** → quay row mới có **password_hash \$2b\$12\$...**

Lời thoại:

“Em đăng ký tài khoản mới **videodemo**. Client gửi **AUTH_SEND_OTP** qua LoadBalancer cổng 9000 — LB peek message đầu, thấy **AUTH_*** nên route về Auth pool. Mã OTP 6 số được hash BCrypt và lưu bảng **email_otp_codes**. Quan sát console Auth log đã gửi mã... Em nhập OTP → tài khoản được tạo, mật khẩu trong DB hoàn toàn không lưu plaintext.”

Cảnh 3.3. OTP rate-limit (~20s)**Thao tác:**

1. Vẫn ở Register form → bấm **Gửi mã OTP** lần 2 ngay lập tức.
2. Quay UI báo:

Vui lòng đợi 1 phút trước khi yêu cầu mã mới

Lời thoại:

“Đây là bằng chứng rate-limit: cùng email trong vòng 1 phút bị từ chối. Đây là tuyến phòng vệ DoS / spam OTP.”

Cảnh 3.4. Đăng nhập sai password + đúng password (~30s)**Thao tác:**

1. Quay sang tab **Đăng nhập**.
2. Nhập **videodemo** / **wrongpass** → click Login → quay UI báo lỗi **Sai tên đăng nhập hoặc mật khẩu**.
3. Nhập **videodemo** / **123456** → Login → vào Lobby.
4. Quay sang phpMyAdmin → refresh **users** → quay cột **last_login_at** đã update timestamp mới.

Lời thoại:

“Đăng nhập sai và đúng đều mất thời gian xấp xỉ nhau (~100ms) vì khi không tồn tại username, server vẫn chạy 1 BCrypt dummy — **constant-time login** chống user-enumeration qua timing attack.”

Cảnh 3.5. Quên mật khẩu (~40s)**Thao tác:**

1. Logout về **LoginForm** → click **Quên mật khẩu**.
2. Nhập email vừa đăng ký → bấm **Gửi mã**.
3. Đợi OTP (qua email hoặc console).
4. Nhập OTP + mật khẩu mới **654321** → submit → UI báo thành công.
5. Login bằng mật khẩu mới → vào Lobby OK.

Lời thoại:

“Quên mật khẩu cũng dùng pipeline OTP tương tự, share cùng modal **OtpVerifyForm**. Single-use: OTP đã dùng cho reset thì không xài lại được — em đã test trong code.”

4. Vẽ realtime — App Logic + Socket + Multi Client (C.1, C.6) — 4:30–9:00**Cảnh 4.1. Chuẩn bị 3 Client cùng phòng (~40s)****Thao tác:**

1. Trong VS, chuột phải **CanvasApp.Client** → Debug → **Start New Instance** × 3 lần (đã có 1 từ phần 3, mở thêm 2).
2. Sắp xếp 3 cửa sổ Client thành **3 cột song song** chiếm hết màn hình (dùng Snap Windows hoặc PowerToys FancyZones).
3. Login lần lượt:
 - Client #1: **demo1** / **123456**
 - Client #2: **demo2** / **123456**
 - Client #3: **demo3** / **123456**
4. Client #1: bấm **Tạo bảng trắng mới** → đặt tên **DEMO-VIDEO** → tick **Public** → bấm Tạo.
5. Quay rõ console **LoadBalancer**:

```
[ROUTE] room <RoomId> claimed by 127.0.0.1:9002 (sniff ROOM_CREATE_RESULT)
```

6. Trong phòng, Client #1 click nút **Sao chép link mời** → quay clipboard / hộp thoại.
7. Client #2 & #3 ở Lobby → paste invite code vào ô **Tham gia bằng mã** → vào cùng phòng.
8. Quay console LB:

```
[ROUTE] room <RoomId> -> 127.0.0.1:9002 (rooms=1, sticky)
```

Lời thoại:

“Em vừa demo cùng lúc **Multi Client + Multi Server room-affinity**: 3 client cùng đi qua LB, nhưng cả 3 đều bị sticky-routing về CanvasServer 9002 vì phòng đã claim ở đó. Đây là điều cần thiết để giữ trạng thái phòng đồng nhất trên 1 server.”

Cảnh 4.2. Demo Pen + đồng bộ realtime (~30s)

Thao tác:

1. Client #1: chọn **Pen**, màu đỏ, độ dày 4 → vẽ chữ ký nguệch ngoạc.
2. Client #2 & #3: thấy nét vẽ xuất hiện gần như tức thì (< 100ms).
3. Trong khi vẽ, quay nhanh console **Canvas Server 9002** → quay log:

```
[ROOM] DRAW_START from demo1 seq=1
[ROOM] DRAW_MOVE  from demo1 seq=2..N
[ROOM] DRAW_END   from demo1
```

Lời thoại:

“Mỗi nét vẽ là 1 **DrawAction**, gửi qua TCP line-delimited JSON. Server gán **SeqNo** đơn điệu bằng **Interlocked.Increment** rồi **BroadcastAsync** cho mọi client trong phòng (trừ sender). 2 client còn lại nhận **BROADCAST_DRAW** và replay đúng nét vẽ.”

Cảnh 4.3. Tour 11 công cụ vẽ — phần Creative (~3 phút)

Mỗi tool quay **1 vòng tay nhanh** (~15s/tool). Vẽ vào canvas chữ to “DEMO” hoặc hình minh hoạ ý nghĩa.

a) Pen + độ dày + bảng màu (~15s)

- Pen, chọn vài màu khác nhau từ palette → vẽ vài nét.

b) Eraser (~10s)

- Chọn Eraser → tẩy 1 phần vừa vẽ. **Nói:** “Eraser dùng **CompositingMode.SourceCopy** — alpha compositing, xoá pixel thay vì blend.”

c) Shapes — Rectangle / Circle / Triangle (~20s)

- Mở dropdown shape → vẽ 1 hình chữ nhật (no fill), 1 hình tròn (fill xanh — bật **chkFill**), 1 tam giác.
- Quay rõ bbox preview khi đang kéo chuột.

d) Line (~10s)

- Vẽ 1 đường thẳng từ góc trên-trái xuống góc dưới-phải.

e) Arrow — 4 biến thể (~20s)

- Mở dropdown arrow → vẽ lần lượt: **Mũi tên đơn**, **Mũi tên hai đầu**, **Mũi tên đứt nét**, **Mũi tên đậm**.
- **Nói:** “4 biến thể dùng **AdjustableArrowCap** + **DashStyle.Dash**.”

f) Text (~25s)

- Chọn Text → click vào canvas → quay caret nhấp nháy.
- Gõ chữ "Nhóm 14" → Enter → text commit.
- Click trúng text vừa gõ → vào edit mode → sửa thành "Nhóm 14 — CanvasApp".
- **Nói:** "Text dùng `Graphics.DrawString` + caret 530ms (đúng chu kỳ Windows). Click trúng text cũ vào lại edit mode — encode nội dung text vào `DrawAction.Type` dạng `text:<nội-dung>`."

g) Fill / Flood Fill (~15s)

- Vẽ trước 1 hình chữ nhật rỗng → chọn Fill, màu vàng → click vào trong hình → toàn vùng được đổ màu.
- **Nói:** "Flood Fill 4-connected, dùng BFS + `Bitmap.LockBits` để truy cập pixel raw — nhanh hơn `GetPixel/SetPixel` khoảng 50 lần."

h) Smart Shape Recognition — Creative ★ (~30s)

- Bật toggle **Smart Shape**.
- Chọn Pen → cố tình **vẽ tay run rẩy 1 hình chữ nhật**.
- Quay rõ overlay đề xuất hiện ra với 2 nút **Accept (Enter) / Reject (Esc)** + hình chuẩn được đề xuất.
- Bấm **Enter** → nét tay biến thành hình chuẩn.
- Lặp lại với hình tròn vẽ tay → Accept.
- Lặp lại với 1 đường thẳng nguệch ngoạc → Accept.
- **Nói:** "Module nhận dạng dùng Chaikin smoothing + Ramer–Douglas–Peucker + Coefficient of Variation cho circle + residual của phương trình ellipse. Mỗi detector $\geq 75\%$ confidence mới đề xuất. Đây là tính năng Creative lấy cảm hứng Excalidraw."

i) Image overlay — Insert/move/resize + sync ★ (~30s)

- Toolbar → nút **Image** → chọn 1 file PNG nhỏ (`test.png` ~100KB).
- Ảnh hiện trên canvas; chọn tool **Select** → kéo ảnh, resize bằng handle góc.
- Quay Client #2 → thấy ảnh xuất hiện và **đồng bộ vị trí khi kéo** (sự kiện `DRAW_IMAGE_TRANSFORM`).
- **Nói:** "Ảnh không rasterize vào bitmap — lưu thành object `CanvasImage` riêng để di chuyển. Mỗi lần move/resize phát `DRAW_IMAGE_TRANSFORM` cho các client khác."

j) Undo / Redo (~15s)

- Bấm Undo 3 lần → 3 action gần nhất biến mất.
- Bấm Redo 3 lần → khôi phục.
- **Nói:** "Command pattern + 2 stack. Mỗi `DRAW_UNDO` gửi `ActionId` lên server, server broadcast cho các peer cùng undo."

k) Zoom & Pan (~15s)

- Lăn chuột giữa: zoom out → zoom in.
- Giữ chuột giữa kéo: pan canvas.
- **Nói:** "Affine transform — công thức giữ cố định điểm dưới con trỏ: $\text{pan}' = \text{mousePos} - (\text{mousePos} - \text{pan}) \times (\text{zoomNew} / \text{zoomOld})$."

Bảng chứng tổng cộng cho block này:

- 3 Client cùng phòng, đồng bộ realtime, không drop nét nào.
- 11 tool đều thao tác được, có visual minh hoạ rõ.

5. I/O File + Network (C.2 — 0.5đ) — 9:00–10:30

Cảnh 5.1. Export canvas → PNG (~25s)

Thao tác:

1. Trong phòng **DEMO-VIDEO** (vẫn còn nội dung vừa vẽ).
2. Menu **Tệp** → **Xuất ảnh** → chọn định dạng **PNG** → đặt tên **demo_video_canvas.png** → Save.
3. Mở file vừa lưu bằng Windows Photos → quay rõ nội dung.

Lời thoại:

“File export gọi `bitmap.Save(path, ImageFormat.Png)`. Đây là I/O File thuần.”

Cảnh 5.2. Import ảnh nền (~20s)

Thao tác:

1. Menu **Tệp** → **Đặt ảnh nền** → chọn 1 ảnh JPG.
2. Quay background canvas đổi.
3. Client #2 thấy background đồng bộ.

Cảnh 5.3. Gửi file qua chat (~30s)

Thao tác:

1. Mở panel chat (bên phải) ở Client #1.
2. Nhập text **Xin chào!** → Enter → quay Client #2, #3 nhận tin.
3. Bấm nút **+** (đính kèm) → chọn 1 file PDF nhỏ ($\leq 2\text{MB}$).
4. Quay rõ bubble chat 🗉 **filename.pdf (12 KB)**.
5. Sang Client #2 → click vào file để tải → quay file download xuất hiện trong Downloads.
6. Quay nhanh console Canvas Server:

```
[CHAT_FILE] demo1 sent <name>.pdf (X bytes)
```

Lời thoại:

“File $\leq 2\text{MB}$ được mã hoá base64 và gửi trong message **CHAT_FILE**. Chat đồng bộ realtime cho mọi client trong phòng. Client kia click tải → file lưu xuống Downloads.”

6. Database persistence (C.3 — 0.5đ) — 10:30–11:30

Cảnh 6.1. Quan sát bảng **canvas_snapshots** + **draw_actions** (~20s)

Thao tác:

1. Quay sang phpMyAdmin → bảng `draw_actions` → quay rõ vài row mới (cột `action_type`, `room_id`, `user_id`, `timestamp`).
2. Bảng `canvas_snapshots` → có ít nhất 1 row nếu đã quá 60s (auto-save). Nếu chưa, đợi thêm hoặc nhấn ép.

Lời thoại:

“Mỗi `DrawAction` được enqueue vào `PersistenceQueue` — `BlockingCollection` chạy 1 background consumer batch INSERT để không nghẽn vòng broadcast. Cứ 60 giây, `AutoSaveService` snapshot toàn bộ canvas dưới dạng GZip + base64 vào `canvas_snapshots`.”

Cảnh 6.2. Restart server vẫn giữ canvas (~40s)**Thao tác:**

1. Tắt Client #2, #3 (đóng cửa sổ).
2. Quay sang **Canvas Server 9002 console** → bấm Ctrl+C để tắt.
3. PowerShell → restart:

```
. \CanvasApp.Server\bin\Debug\CanvasApp.Server.exe 9002
```

4. Đợi banner `CANVAS SERVER listening on :9002`.
5. Client #1 (vẫn mở): bấm **Thoát phòng** → quay lại Lobby → vào lại phòng `DEMO-VIDEO`.
6. Quay rõ canvas khôi phục đầy đủ nét vẽ + ảnh + text.

Lời thoại:

“Em vừa tắt CanvasServer rồi bật lại. Client join lại phòng cũ — `RoomManager.EnsureCanvasLoaded` đọc snapshot mới nhất + replay delta từ bảng `draw_actions`. Canvas y nguyên — tính bền vững dữ liệu.”

7. Multi-thread (C.4 — 0.5đ) — 11:30–12:15**Cảnh 7.1. Code evidence 4 pattern (~30s)**

Thao tác: Mở 4 file nhanh trong VS, mỗi file dừng ~7s:

File	Highlight
<code>CanvasApp.Server/Program.cs</code> (đoạn <code>HandleClient</code>)	<code>Task.Run(() => HandleClient(client))</code>
<code>CanvasApp.Common/DataAccess/PersistenceQueue.cs</code>	<code>BlockingCollection</code> + 1 consumer task
<code>CanvasApp.Server/AutoSaveService.cs</code>	<code>System.Timers.Timer</code> interval 60s
<code>CanvasApp.Server/RoomManager.cs</code>	<code>ConcurrentDictionary</code> + lock + <code>Interlocked</code>

Lời thoại:

"4 pattern đa luồng: (1) async/await + 1 Task cho mỗi client, (2) producer-consumer write-behind queue cho DB, (3) Timer định kỳ cho auto-save + health-check + peer heartbeat, (4) collections thread-safe + lock + Interlocked cho atomic counters."

Cảnh 7.2. 3 client cùng vẽ — không lag (~15s)

Thao tác: Quay nhanh 3 client cùng vẽ 1 lúc — không miss nét, không crash.

Bằng chứng cần show: console Canvas có batch log:

```
[PERSISTENCE] batched 47 actions in 12 ms
```

8. Cryptography (C.8 — 0.5đ) — 12:15–14:00

Cảnh 8.1. BCrypt hash trong DB (~15s)

Thao tác: Quay phpMyAdmin → `users.password_hash $2b$12$...` (đã làm ở phần 2 — chỉ nhắc lại).

Cảnh 8.2. Token HMAC tamper (~50s)

Thao tác:

1. Trong VS, set breakpoint ở `LoginForm.LoginAsync` ngay sau khi nhận token, hoặc đơn giản: dùng `Debug → Watch` để copy `Session.Token`.
2. Quay cửa sổ Watch → token có format `<base64payload>.<base64signature>`.
3. Trong file `Session.cs`, **tạm sửa 1 ký tự** ở phần signature (hard-code token cũ).
4. Restart Client → login → server reject với message `Token không hợp lệ`.
5. Revert sửa.

Lời thoại:

"Token = `base64(payload).base64(HMAC-SHA256(payload, secret))`. Khi em chỉnh 1 ký tự ở signature, `FixedTimeEquals` thấy MAC không khớp và reject. Đây là chứng minh **chống forge token**."

💡 **Phương án nhanh hơn:** chỉ cần show code `UserStore.cs VerifyToken` + nói "Em đã test scenario này" + cắt sang phần AES nếu cần tiết kiệm thời gian.

Cảnh 8.3. AES-256-CBC + HMAC demo (~50s)

Thao tác:

1. Mở **LINQPad** (hoặc Console project tạm).
2. Paste đoạn code AES từ `Docs/DEMO_GUIDE.md §D.2`.
3. Chạy → quay rõ output:

```
Encrypted (124 chars base64):
yIAjVxqxJK6Xq7M9...+8h2QYpZCk3W0eaPfpg==
```

```
Decrypted:
{"type":"AUTH_LOGIN","data":{"Username":"ndln","Password":"secret"}}
```

✅ Tampering caught: HMAC verification failed (tampered or wrong key)

4. Chạy thêm 1 lần nữa → quay **base64 khác** (IV random mỗi lần encrypt).

Lời thoại:

“Module AES-256-CBC + HMAC-SHA256 theo pattern Encrypt-then-MAC. Định dạng: **[IV 16B][CT][HMAC 32B]** → base64. Em sửa 1 ký tự ciphertext → HMAC verify fail trước khi decrypt — bảo vệ integrity. Module đã sẵn sàng nhưng cố tình **không bật ở wire protocol** để Wireshark sau đây có thể đọc JSON thật.”

9. Wireshark — chứng minh traffic thật trên dây (~14:00–15:30)

Cảnh 9.1. Start capture + filter (~20s)

Thao tác:

1. Mở Wireshark → chọn **Adapter for loopback traffic capture** → bấm Start (vây cá mập).
2. Display Filter: **tcp.port == 9000**.

Cảnh 9.2. Capture 1 login flow (~30s)

Thao tác:

1. Mở Client #4 mới → login **demo1**.
2. Quay sang Wireshark → click 1 packet → **Right-click** → **Follow** → **TCP Stream**.
3. Quay rõ JSON:

```
{"type":"AUTH_LOGIN","data":
{"Username":"demo1","Password":"123456"},"token":null}
{"type":"AUTH_LOGIN_RESULT","data":{"Success":true,"Token":"eyJ...","User":
{...}}}
```

Lời thoại:

“Đây là **JSON plaintext qua TCP** — chứng minh giao thức thật. Password **123456** đang xuất hiện trên dây vì AES đang tắt; em chỉ cần uncomment 2 dòng ở 5 file là cả lớp message được mã hoá. Trong báo cáo em đã giải thích trade-off này.”

Cảnh 9.3. Capture DRAW flow (~25s)

Thao tác:

1. Đổi filter: `tcp contains "DRAW_"`.
2. Vẽ 1 nét bằng Client #1.
3. Quay loạt packet: `DRAW_START` → `DRAW_MOVE` × N → `DRAW_END` rồi `BROADCAST_DRAW` ngược lại các client khác.

Lời thoại:

"Đây là vòng đời 1 nét vẽ end-to-end: client gửi START, các MOVE intermediate, rồi END. Server broadcast về dạng `BROADCAST_DRAW`."

Cảnh 9.4. Capture peer mesh (~20s)

Thao tác:

1. Đổi filter: `tcp.port == 9102 or tcp.port == 9103`.
2. Trên Client #2 (nếu đang ở Canvas khác), gửi 1 chat — không thì tạo 1 phòng mới đẩy về Canvas 9003.
3. Quay packet `PEER_RELAY` chứa inner `CHAT_MESSAGE`.

Lời thoại:

"Đây là peer mesh giữa 2 CanvasServer — khi 2 client cùng phòng nhưng kết nối khác server, server-A relay event qua server-B qua peer channel."

10. Load Balancing — 6 bằng chứng (C.11 — 1.0đ) — 15:30–18:30

Quan trọng nhất — chiếm 1.0đ riêng. Quay 6 bằng chứng tuần tự.

Cảnh 10.1. Bằng chứng #1 — Banner 2+2 (~10s)

- Quay lại console **LoadBalancer** → quay banner `Auth pool: 2, Canvas pool: 2` (đã set ở phần 2).

Cảnh 10.2. Bằng chứng #2 — Health Check 3-strike DOWN (~40s)

Thao tác:

1. Quay sang PowerShell đang chạy `CanvasApp.Server.exe 9002` (server gốc Canvas đầu) → bấm **Ctrl+C** → tắt.
2. Quay console LB → đợi ~15s:

```
[HEALTH] Canvas 127.0.0.1:9002 probe 1/3 failed (Connection refused)
[HEALTH] Canvas 127.0.0.1:9002 probe 2/3 failed (Connection refused)
[HEALTH] Canvas 127.0.0.1:9002 -> DOWN (3 fails: Connection refused)
```

Lời thoại:

"LB không mark DOWN ngay lần fail đầu — phải fail liên tiếp 3 lần (~15s) mới DOWN. Đây là **3-strike pattern** chống flap khi mạng tạm chập."

Cảnh 10.3. Bằng chứng #3 — UP (recovered) (~25s)

Thao tác:

1. PowerShell → restart:

```
.\CanvasApp.Server.exe 9002
```

2. Quay console LB → ≤5s sau:

```
[HEALTH] Canvas 127.0.0.1:9002 -> UP (recovered)
```

Cảnh 10.4. Bằng chứng #4 — Auth round-robin (~30s)

Thao tác:

1. Đóng hết Client cũ.
2. Mở Client #1 → login **demo1** → quan sát console LB:

```
[LB] 127.0.0.1:xxxx -> Auth 127.0.0.1:9001 [UP] (first=AUTH_LOGIN)
```

3. Mở Client #2 → login **demo2** → quan sát:

```
[LB] 127.0.0.1:yyyy -> Auth 127.0.0.1:9011 [UP] (first=AUTH_LOGIN)
```

Lời thoại:

“2 login liền kề đi 2 AuthServer khác — bằng chứng round-robin.”

Cảnh 10.5. Bằng chứng #5 — Canvas room-affinity stickiness (~50s)

Thao tác:

1. Client #1: vào Lobby → **Tạo phòng AFFINITY-TEST**.
2. Quay console LB:

```
[ROUTE] room <RoomId> claimed by 127.0.0.1:9002 (sniff ROOM_CREATE_RESULT)
```

3. Trong phòng → bấm **Sao chép link mời** → copy.
4. Client #2 (instance mới hoàn toàn): login **demo2** → ngay ô **Tham gia bằng mã** paste invite → submit.
5. Quay console LB:

```
[ROUTE] room <RoomId> -> 127.0.0.1:9002 (rooms=1, sticky)
[LB] ... -> Canvas 127.0.0.1:9002 ... (first=ROOM_JOIN_BY_CODE)
```

6. Vào phòng → vẽ → đồng bộ → bằng chứng routing đúng.

Lời thoại:

“LB ‘sniff’ message đầu của connection — gặp **ROOM_JOIN_BY_CODE** chứa RoomId → look up bảng route → đẩy Client #2 vào cùng CanvasServer với Client #1. Đây là room-affinity, đảm bảo trạng thái phòng nằm 1 nơi.”

Cảnh 10.6. Bằng chứng #6 — Failover (~40s)

Thao tác:

1. 2 client đang ở phòng **AFFINITY-TEST** trên Canvas 9002.
2. Tắt Canvas 9002 (Ctrl+C trong PowerShell tương ứng).
3. Quay console LB → log 3-strike DOWN.
4. Quay Client → hiển thị **Mất kết nối, đang thử lại...** → reconnect logic kích.
5. Client #1: tạo phòng mới **FAILOVER-TEST** → LB route về **9003**:

```
[LB] ... -> Canvas 127.0.0.1:9003 ...
```

6. Restart 9002 → LB log **UP (recovered)**.

Lời thoại:

“Khi 9002 chết, LB tự kick connection ra. Client reconnect — LB chỉ còn 1 Canvas UP nên dồn vào 9003. Khi 9002 sống lại, pool đầy đủ trở lại.”

11. Demo LAN / Demo Internet (C.9, C.10 — 1.0đ) — 18:30–20:30

Hai mục này **bắt buộc cần 2 máy thật**. Nếu chỉ có 1 máy: quay riêng video phụ trước với laptop thứ 2 / máy bạn, rồi cắt vào video chính. Hoặc dùng máy ảo VMware/VirtualBox trên cùng máy host (tốc độ chậm hơn nhưng vẫn chứng minh được).

Cảnh 11.1. Demo LAN — 2 máy cùng Wi-Fi (~50s)

Thao tác máy A (server):

1. PowerShell → **ipconfig** → quay rõ **IPv4 Address** (vd **192.168.1.42**).
2. (Nếu cần) tắt Windows Firewall hoặc add Inbound Rule cho 9000–9011 + 9102/9103.
3. Đảm bảo 5 server đang chạy.

Thao tác máy B (client):

1. Trong source Client (đã build sẵn cho máy B), file **Session.cs**:

```
public const string LB_HOST = "192.168.1.42";
```

2. Build lại → chạy Client.
3. Login **demo1** → join phòng.

Quay split-screen 2 màn hình A/B:

- Máy A vẽ → máy B thấy realtime.
- `netstat -ano | findstr :9000` trên máy A → thấy connection **ESTABLISHED** với IP máy B.

Lời thoại:

"2 máy cùng Wi-Fi, server bind **IPAddress.Any** nên máy ngoài LAN kết nối được. Đây là mục Demo LAN."

Cảnh 11.2. Demo Internet — qua VPN ảo Radmin (~50s)

Thao tác:

1. Cài **Radmin VPN** trên cả 2 máy → join cùng network → quay UI Radmin show IP **26.x.x.x**.
2. Máy A: `ipconfig` → tìm interface **Radmin VPN** → quay IP.
3. Máy B: sửa `Session.cs` → `LB_HOST = "26.x.x.x"` → rebuild → chạy Client từ một mạng **khác** (4G/Wi-Fi khác).
4. Login → join phòng → vẽ → đồng bộ.

Lời thoại:

"2 máy ở 2 mạng Internet khác nhau, kết nối qua VPN ảo Radmin. Đây là mục Demo Internet — không cần port forward router."

12. Bonus — owner xoá phòng / cleanup ref-counting (~20:30–21:00)

Cảnh 12.1. Owner xoá phòng — sync sang server khác (~25s)

Thao tác:

1. Tạo 2 phòng ở 2 Canvas khác nhau (round-robin).
2. Client #1 (owner phòng trên 9002) → menu → **Xoá phòng** → confirm.
3. Quay console Canvas 9003:

```
[PEER_ROOM_DELETE] room X removed from local state
```

4. Client #2 ở Lobby (trên Canvas 9003) → đợi $\leq 3s$ polling → quay phòng biến mất khỏi danh sách.

Cảnh 12.2. Cleanup ref-counting (~15s)

Thao tác:

1. Tắt cả client trong phòng leave hết.
2. Quay console LB:

```
[ROUTE] room <RoomId> freed from 127.0.0.1:9002 (rooms=0)
```

13. Wrap-up & checklist — 21:00–22:00

Cảnh 13.1. Slide tổng kết (~30s)

Hiển thị: Slide checklist 11 mục rubric, từng dòng tick xanh  + screenshot/clip nhỏ minh hoạ ở phải.

#	Mục	Điểm	✓	Cảnh đã quay
1	App Logic + Socket Logic	5.0		Cảnh 4.2, 9.3
2	I/O File + Network	0.5		Cảnh 5.1–5.3
3	Database	0.5		Cảnh 6.1–6.2
4	Thread / Đa luồng	0.5		Cảnh 7.1–7.2
5	Sign up / Sign in / OTP / Forgot	0.5		Cảnh 3.1–3.5
6	Multi Client	0.5		Cảnh 4.1–4.3
7	Multi Server	0.5		Cảnh 2.3, 2.4
8	Cryptography	0.5		Cảnh 8.1–8.3
9	Demo LAN	0.5		Cảnh 11.1
10	Demo Internet (VPN)	0.5		Cảnh 11.2
11	Load Balancing	1.0		Cảnh 10.1–10.6
Σ		10.0		

Creative ★ + UI:

- Smart Shape Recognition (Cảnh 4.3.h)
- Image overlay + multi-handle resize (Cảnh 4.3.i)
- Infinite canvas ([EnsureCanvasCovers](#)) — show qua thao tác pan ra xa.
- Forgot Password OTP (Cảnh 3.5)
- Peer mesh + cross-server sync (Cảnh 9.4, 12.1)

Cảnh 13.2. Lời kết (~30s)

Lời thoại:

“CanvasApp đáp ứng đầy đủ 11 mục rubric với cấu trúc 5 service tách rời, mọi I/O đi qua TCP plain [System.Net.Sockets](#), mật khẩu băm BCrypt cost 12, token ký HMAC-SHA256, và module AES-256-CBC + HMAC sẵn sàng. Cảm ơn thầy/cô đã theo dõi — nhóm 14 xin nhận câu hỏi.”

Hiển thị slide cuối: repo GitHub (nếu có) + email liên hệ + tên 3 thành viên.

Phụ lục A — Plan B & Troubleshooting khi quay

A.1. Trouble: Wireshark loopback không thấy gì

- Cài lại **Npcap** với tick **WinPcap API-compatible Mode**.
- Stop & Start capture lại 1 lần.

A.2. Trouble: F5 báo lỗi build hoặc **Common.dll** bị lock

```
Get-Process CanvasApp.* -ErrorAction SilentlyContinue | Stop-Process -Force
```

Rồi Rebuild solution.

A.3. Trouble: Health check không log DOWN

- Có thể CanvasServer chưa thật sự chết — xác nhận bằng:

```
netstat -ano | findstr :9002
```

- Nếu vẫn còn → kill process bằng PID.

A.4. Trouble: Room-affinity không kick in

- Phải dùng **Join by Invite Code** (không phải click card trong Lobby).
- Code lobby gửi **ROOM_LIST** trước → LB không peek được RoomId → fallback least-loaded.

A.5. Trouble: 2 máy LAN không thấy nhau

- Tắt firewall hoặc add Inbound Rule cho **9000, 9001, 9002, 9003, 9011, 9102, 9103**.
- Confirm cùng subnet bằng **ipconfig** trên 2 máy.

A.6. Trouble: OTP không gửi đến email

- Check **CanvasApp.AuthServer/App.config** cấu hình SMTP.
- Plan B: dev mode in OTP plaintext ra **console** — đủ làm bằng chứng.

A.7. Plan B: chỉ có 1 máy duy nhất

- **Demo LAN:** dùng máy ảo VMware/VirtualBox cài Win10 → cài Client → kết nối ra host bằng IP của vEthernet (**192.168.x.x**).
- **Demo Internet:** quay riêng clip ngắn ở nhà trước (mượn laptop bạn), cắt vào video chính + caption "Quay riêng tại ... ngày ...".

A.8. Plan B: thiếu thời gian — cắt cảnh nào?

- Bỏ Cảnh 12 (bonus), Cảnh 8.2 token tamper (chỉ show code).
- Gộp Cảnh 5.2 import + 5.3 chat thành 1.
- Smart Shape chỉ vẽ 1 hình (vd hình tròn) thay vì 3.

Phụ lục B — Voice-over script tổng (rút gọn để in ra giấy)

[0:00] CanvasApp – bảng vẽ cộng tác realtime trên TCP. Nhóm 14, NT106.

[1:00] MySQL chạy. F5 mở 3 console: Auth, Canvas, LB. CLI args mở 2 instance phụ. Tổng 5 server. netstat xác nhận 7 port LISTENING.

[2:30] Đăng ký user mới: OTP qua email, BCrypt cost 12 trong DB. Demo rate-limit 1/phút. Constant-time login. Forgot password.

[4:30] 3 client cùng phòng, room-affinity sticky về 9002. Demo 11 tool: Pen, Eraser, 4 shapes, Arrow 4 biến thể, Text caret, Flood fill BFS, Smart Shape recognition (Creative), Image overlay (Creative), Undo/Redo, Zoom/Pan.

[9:00] Export PNG, import ảnh nền, gửi PDF qua chat realtime.

[10:30] Bảng draw_actions + canvas_snapshots. Tắt Canvas → restart → canvas khôi phục y nguyên.

[11:30] 4 pattern thread: Task per client + BlockingCollection + Timer + ConcurrentDictionary.

[12:15] BCrypt \$2b\$12\$, token HMAC-SHA256 tamper reject, AES-256-CBC + HMAC encrypt-then-MAC demo.

[14:00] Wireshark JSON plaintext qua TCP. Capture login. Capture DRAW. Capture PEER_RELAY.

[15:30] LB: banner 2+2, health 3-strike DOWN, UP recovered, Auth round-robin, Canvas room-affinity, failover.

[18:30] Demo LAN 2 máy cùng Wi-Fi. Demo Internet qua Radmin VPN.

[20:30] Bonus: peer broadcast khi xóa phòng, ref-counting cleanup.

[21:00] Checklist 11/11 . Lời kết.

Phụ lục C — Danh sách screenshot/file đính kèm cùng video

Nộp kèm trong cùng folder với video:

1. [assets/01_pool_2plus2.png](#) — Banner LB Auth/Canvas pool 2+2.
2. [assets/02_health_down.png](#) — Log 3-strike DOWN.
3. [assets/03_health_up.png](#) — Log UP (recovered).
4. [assets/04_room_affinity.png](#) — Log sniff ROOM_CREATE_RESULT + sticky.
5. [assets/05_bcrypt_hash.png](#) — phpMyAdmin show \$2b\$12\$....
6. [assets/06_otp_hash.png](#) — phpMyAdmin show email_otp_codes.code_hash.
7. [assets/07_aes_demo.png](#) — Output LINQPad AES encrypt + tamper.
8. [assets/08_wireshark_login.png](#) — Follow TCP Stream AUTH_LOGIN.
9. [assets/09_wireshark_draw.png](#) — DRAW_START/MOVE/END flow.
10. [assets/10_peer_relay.png](#) — Wireshark peer mesh.
11. [assets/11_netstat.png](#) — netstat 7 port LISTENING.
12. [assets/12_canvas_persist.png](#) — Trước/sau restart, canvas giữ nguyên.
13. [assets/13_smart_shape.png](#) — Suggestion overlay (Creative).
14. [assets/14_lan_demo.jpg](#) — Ảnh chụp 2 máy thật cạnh nhau.

15. [assets/15_vpn_demo.png](#) — UI Radmin VPN + canvas đồng bộ.

16. [pcap/canvasapp_demo.pcapng](#) — File capture Wireshark export.

Hết kịch bản. In ra giấy A4, gấp đôi để cạnh bàn khi quay. Quay 2 take: take 1 thô (~25p), take 2 sau khi rút kinh nghiệm (~18p). Edit cắt bớt phần ấp úng + thêm caption highlight các log key ([DOWN](#), [UP \(recovered\)](#), [ROUTE](#), [sticky](#), [BCrypt \\$2b\\$12\\$, HMAC verification failed](#)).